Chapter 14 Redes neurais

▶ Carregar bibliotecas e configurações comuns

▶ Carregar bibliotecas adicionais

Os modelos de previsão tradicionais, como ARIMA e ETS, tratam cada série temporal separadamente, criando modelos individuais. Esses são modelos locais. Eles podem não ser adequados para previsões em larga escala, pois a manutenção de modelos individuais é impraticável. Além disso, o uso de covariáveis exógenas em modelos tradicionais geralmente requer elaboração e ajuste cuidadosos.

As redes neurais (RNs) podem solucionar esses problemas. Elas aprendem uma representação compactada dos dados de entrada, permitindo um único modelo para diversas séries temporais. Isso possibilita o aprendizado de interações complexas entre várias fontes de dados. Esses são modelos globais. Eles simplificam a previsão, eliminando a necessidade de modelos separados e recursos definidos manualmente. As redes neurais têm uma longa história que remonta à década de 1960, mas só recentemente se popularizaram para previsão em laboratórios especializados. Exemplos incluem Amazon DeepAR (Salinas et al. 2019) , Google TFT (Lim et al. 2021) e Nixtla NHITS (Challu et al. 2023) .

Este capítulo começa com uma breve introdução à previsão com redes neurais na Seção 14.1 . Em seguida, apresenta uma arquitetura central de rede neural, o Perceptron Multicamadas (MLP), na Seção 14.2 . Depois, aborda arquiteturas de redes neurais mais complexas na Seção 14.3 . As seções posteriores (14.4 a 14.7) explicam como fazer previsões com redes neurais usando a biblioteca NeuralForecast .

14.1 Previsão neural

Ao usar redes neurais para previsão, nosso objetivo é aprender uma representação (comprimida) dos nossos dados de entrada (Bengio, Courville e Vincent 2013) . Matematicamente, uma rede neural é uma função. $f_\theta : X \mapsto Y$, com X o espaço de entrada/características e Y o espaço da variável dependente. Consideramos o cenário com $X = y_{[0:t]}, x_{[0:t+h]}$ e $Y = y_{[t+1:t+h]}$, onde h é o horizonte de previsão, y é a série temporal alvo, e x são covariáveis exógenas. A tarefa de

previsão consiste em estimar a seguinte distribuição condicional:

$$P\left(y_{[t+1:t+h]} \mid y_{[0:t]}, x_{[0:t+h]}\right) = f_\theta\left(y_{[0:t]}, x_{[0:t+h]}\right). \tag{14.1}$$

Podemos aprender a função f_θ minimizando um objetivo de otimização, geralmente chamado de *função de perda* . Podemos formalizar o objetivo de otimização da seguinte forma:

$$L = f\left(Y, \hat{Y}_\theta\right), \tag{14.2}$$

em que $\hat{Y}_\theta$ denota as previsões produzidas pela nossa rede neural. f_θ Um objetivo de otimização frequentemente usado em previsões com redes neurais é o *Erro Médio Absoluto (MAE)* (detalharemos alguns objetivos de otimização comuns na Seção 14.5):

$$\text{MAE} = \frac{1}{N}\sum_{i=0}^{N} \mid Y_{eu} - \hat{Y}_{i,\theta}\mid. \tag{14.3}$$

Obtemos a melhor previsão quando o objetivo da otimização é minimizado, o que, no caso do MAE, significa a previsão. $\hat{Y}_{i,\theta}$ igual ao valor observado real Y_{eu} O procedimento mais utilizado para minimizar a função objetivo de otimização é chamado de *Descida do Gradiente* (DG). Na DG, atualizamos iterativamente os parâmetros treináveis da rede neural utilizando a derivada da função objetivo de otimização em relação aos parâmetros treináveis, ou seja,

$$\theta_{i+1} \leftarrow \theta_{eu} - \alpha\frac{\delta L}{\delta \hat{Y}_\theta}, \tag{14.4}$$

onde α denota a *taxa de aprendizado* , um *hiperparâmetro* que nos permite ajustar a velocidade com que a rede aprende a representação dos dados de entrada. Discutiremos a otimização desses *hiperparâmetros* na Seção 14.7 . A ideia principal é que, se realizarmos um número suficiente de iterações do GD, continuaremos atualizando nossos parâmetros. θ da rede neural em uma direção que minimize o objetivo de otimização.

Em resumo,

- Ao fazer previsões com redes neurais, nosso objetivo é aprender uma representação (comprimida) dos nossos dados de entrada;
- Podemos aprender essa representação minimizando um objetivo de otimização, ou *função de perda* ;
- O objetivo da otimização é geralmente minimizado iterativamente usando o *método do Gradiente Descendente (GD)* ;
- O procedimento de otimização envolve *hiperparâmetros* , que são parâmetros de ajuste que controlam o processo de aprendizagem.

14.2 Perceptron multicamadas

Uma rede neural pode ser vista como uma rede de "neurônios" organizados em camadas. Os preditores (ou entradas) formam a camada inferior, e as previsões (ou saídas) formam a camada superior. Também podem existir camadas intermediárias contendo "neurônios ocultos".

As redes mais simples não contêm camadas ocultas e são equivalentes a regressões lineares. A Figura 14.1 mostra a versão em rede neural de uma regressão linear com quatro preditores. Referimo-nos aos coeficientes desses preditores como “pesos” ou “parâmetros”. As previsões são obtidas por uma combinação linear das entradas. Os pesos são selecionados na estrutura da rede neural usando o método do Gradiente Descendente, que minimiza uma função de perda como o Erro Médio Absoluto (MAE) ou o Erro Quadrático Médio (MSE). Naturalmente, neste exemplo simples, podemos usar a regressão linear, que minimiza o MSE e é um método muito mais eficiente para treinar o modelo.

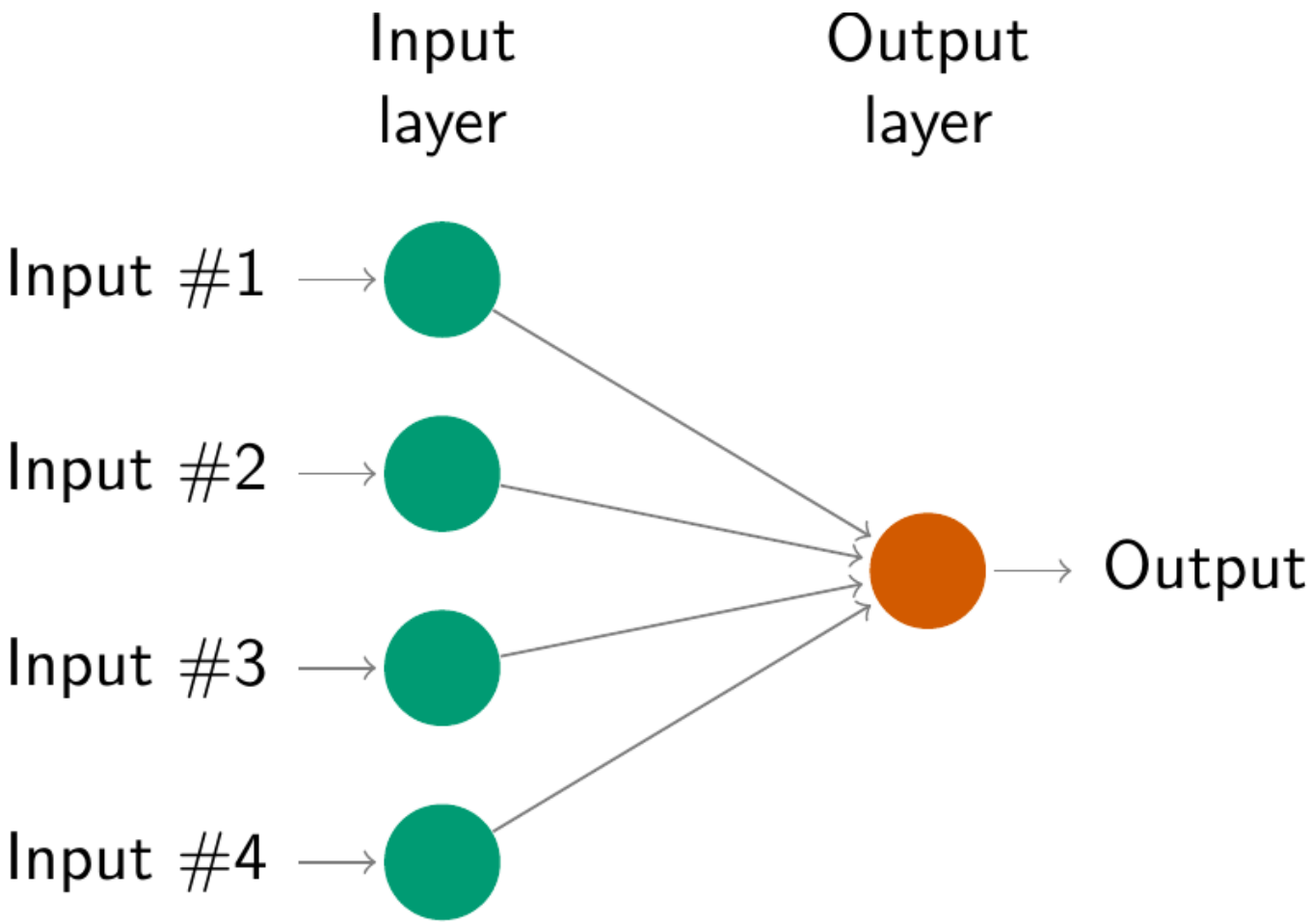

Figura 14.1: Uma rede neural simples equivalente a uma regressão linear.

Ao adicionarmos uma camada intermediária com neurônios ocultos, a rede neural torna-se não linear. Um exemplo simples é mostrado na Figura 14.2 .

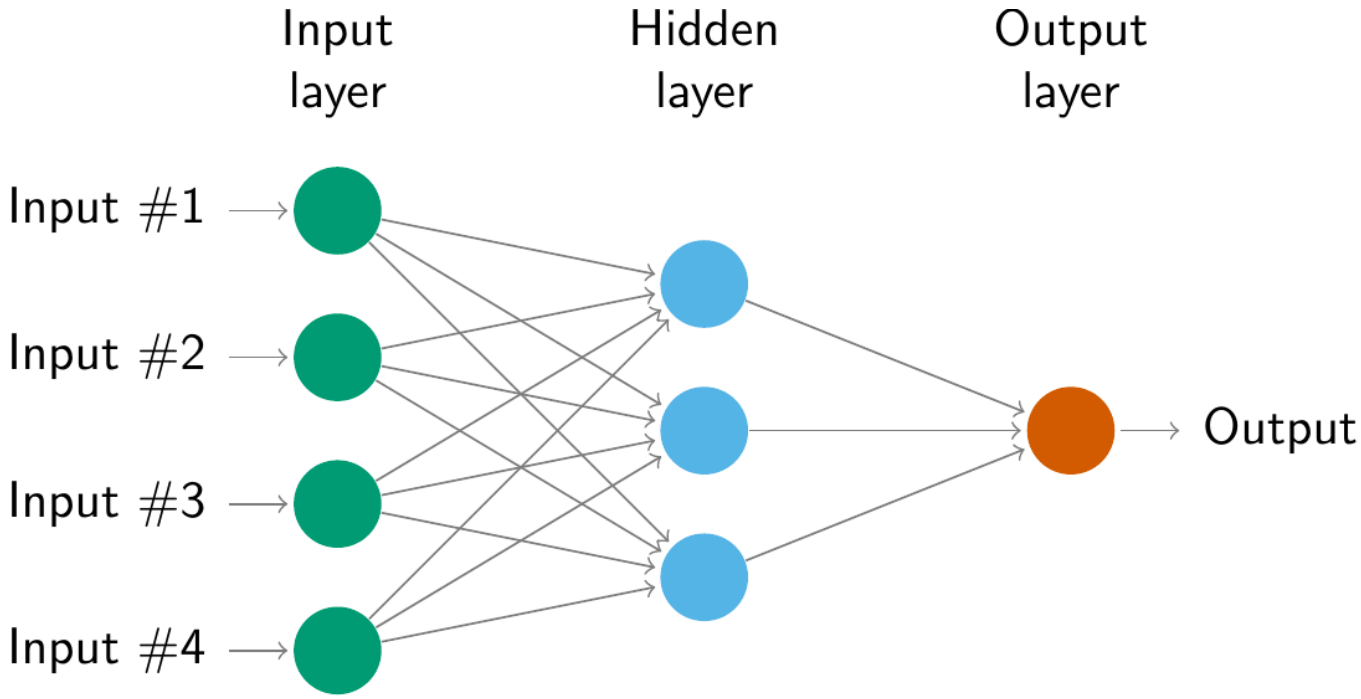

Figura 14.2: Uma rede neural com quatro entradas e uma camada oculta com três neurônios ocultos.

Isso é conhecido como *Perceptron Multicamadas (MLP)* , onde cada camada de nós recebe entradas das camadas anteriores. As saídas dos nós em uma camada são entradas para a próxima camada. As entradas de cada nó são combinadas usando uma combinação linear ponderada. O resultado é então modificado por uma função não linear antes de ser emitido. Por exemplo, as entradas de cada neurônio oculto na Figura 14.2 são combinadas linearmente para dar

$$z_j = b_j + \sum_{i=1}^{4} c_{eu\,,\,j} x_{eu}.$$

In the hidden layer, this is then modified using a nonlinear function such as a sigmoid,

$$s(z) = \frac{1}{1+e^{-z}},$$

to give the input for the next layer. This tends to reduce the effect of extreme input values, thus making the network somewhat robust to outliers.

The parameters b_1, b_2, b_3 and $w_{1,1}, \ldots, w_{4,3}$ are "learned" (or estimated) from the data. The weights are initialized with random values, and are subsequently iteratively updated using Gradient Descent as described in Section 14.1. Consequently, there is an element of randomness in the predictions produced by a neural network.

The number of hidden layers and the number of nodes in each hidden layer are *hyperparameters* that must be specified in advance when trying to make predictions.

Example: Forecasting air passengers with an MLP

The AirPassengers dataset contains data on the number of US airline passengers from 1949 to 1960 for two different airlines. We split the data into a training and test set, where we use the last 12 months for testing. We use an MLP that takes in the last 24 months of observations (`input_size = 24`), and the task is to forecast the next 12 months (horizon `h=12`).

```
# We use the last 12 months (1960) for testing
test_mask = AirPassengersPanel["ds"] >= "1960"
Y_train_df = AirPassengersPanel[~test_mask]
Y_test_df = AirPassengersPanel[test_mask].reset_index(drop=True)
# Instantiate the MLP
model = MLP(h=12, input_size=24, scaler_type="robust")
# Instantiate the forecaster
nf = NeuralForecast(models=[model], freq="M")
# Fit the model to the training set
nf.fit(df=Y_train_df)
# Generate predictions
forecasts = nf.predict()
```

We can verify in Figure 14.3 that the MLP is able to predict the test set for both Airlines fairly accurately.

```
# Plot the predictions and the training data
fig, axes = plt.subplots(2, sharex=True)
plot_series(AirPassengersPanel, forecasts, xlabel="", ylabel="",
    palette="black_and_blue", rm_legend=False, ax=axes)
axes[-1].set(xlabel="Month [1M]")
fig.suptitle("Number of passengers for different airlines", x=.54)
fig.supylabel("Passengers")
fig
```

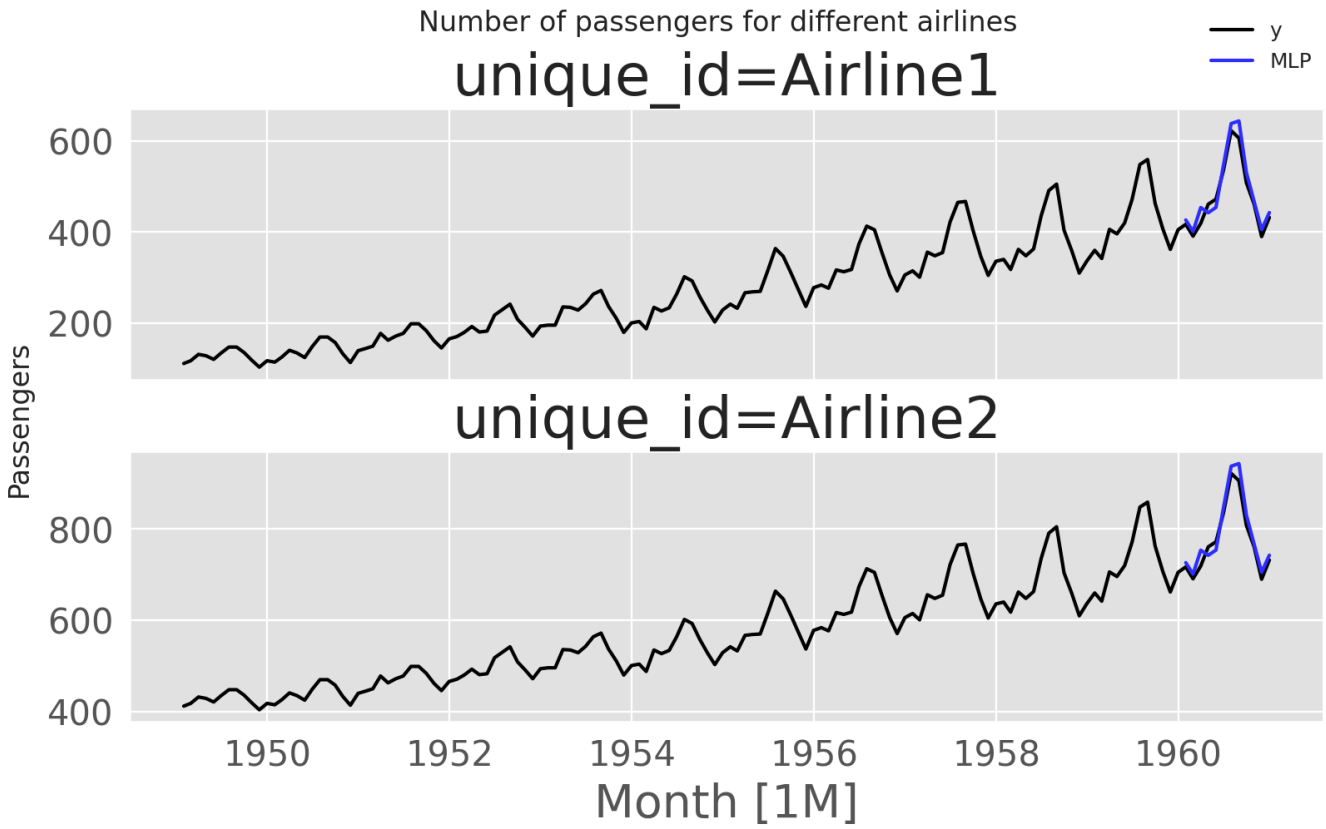

Figure 14.3: AirPassengers MLP example.

14.3 Modern neural network architectures

The *MLP* set out previously is one of the simplest neural network architectures. Nowadays, there is a vast amount of research into the best neural network architectures for forecasting. *NeuralForecast* provides access to many of these modern architectures, listed in Table 14.1.

Table 14.1: Table of neural forecasting methods available in NeuralForecast.

	Model	Family	Univariate / Multivariate	Forecast Type	Exogenous
0	Autoformer	Transformer	Univariate	Direct	F
1	BiTCN	CNN	Univariate	Direct	F/H/S
2	DeepAR	RNN	Univariate	Recursive	F/S
3	DeepNPTS	MLP	Univariate	Direct	F/H/S
4	DilatedRNN	RNN	Univariate	Recursive	F/H/S
5	FEDformer	Transformer	Univariate	Direct	F
6	GRU	RNN	Univariate	Recursive	F/H/S

	Model	Family	Univariate / Multivariate	Forecast Type	Exogenous
7	HINT	Any	Both	Both	F/H/S
8	Informer	Transformer	Multivariate	Direct	F
9	iTransformer	Transformer	Multivariate	Direct	-
10	KAN	KAN	Univariate	Direct	-
11	LSTM	RNN	Univariate	Recursive	F/H/S
12	MLP	MLP	Univariate	Direct	F/H/S
13	MLPMultivariate	MLP	Multivariate	Direct	F/H/S
14	NBEATS	MLP	Univariate	Direct	-
15	NBEATSx	MLP	Univariate	Direct	F/H/S
16	NHITS	MLP	Univariate	Direct	F/H/S
17	NLinear	MLP	Univariate	Direct	-
18	PatchTST	Transformer	Univariate	Direct	-
19	RNN	RNN	Univariate	Recursive	F/H/S
20	SOFTS	MLP	Multivariate	Direct	-
21	StemGNN	GNN	Multivariate	Direct	-
22	TCN	CNN	Univariate	Recursive	F/H/S
23	TFT	Transformer	Univariate	Direct	F/H/S
24	TiDE	MLP	Univariate	Direct	F/H/S
25	TimeMixer	MLP	Multivariate	Direct	-
26	TimeLLM	LLM	Univariate	Direct	-
27	TimesNet	CNN	Univariate	Direct	F
28	TSMixer	MLP	Multivariate	Direct	-
29	TSMixerx	MLP	Multivariate	Direct	F/H/S
30	VanillaTransformer	Transformer	Univariate	Direct	F

In Table 14.1, we distinguish the following columns:

1. **Model**: The model name.
2. **Family**: The main neural network architecture underpinning the model. As you can see, there are many contemporary models that employ a variation of the *MLP* described above.

Other common architecture families are *Convolutional Neural Network (CNN)*, *Transformers*, and *Recurrent Neural Network (RNN)*. It is beyond the scope of this book to explain all these building blocks in detail, and the interested reader is referred to the documentation of NeuralForecast for further explanation of each architecture.

3. **Univariate / Multivariate**: A multivariate model explicitly models the interactions between multiple time series in a dataset and will provide predictions for multiple time series concurrently. In contrast, a univariate model trained on multiple time series implicitly models interactions between multiple time series and provides predictions for single time series concurrently. Multivariate models are typically computationally expensive and empirically do not necessarily offer better forecasting performance compared to using a univariate model.
4. **Forecast Type**: Direct forecast models are models that produce all steps in the forecast horizon at once. In contrast, recursive forecast models predict one-step ahead, and subsequently use the prediction to compute the next step in the forecast horizon, and so forth. Direct forecast models typically suffer less from bias and variance propagation as compared to recursive forecast models, whereas recursive models can be computationally less expensive.
5. **Exogenous**: Whether the model accepts exogenous variables. This can be exogenous variables that contain information about the past and future (F), about the past only (*historical*, H), or that contain static information (*static*, S).

We turn to the `AirPassengers` dataset once again. Now, we train it using two more modern architectures, for example `NHITS()` and `RNN()`. We find that the results are comparable to the results we obtained using the relatively simple MLP architecture, which is due to the relative simplicity of the AirPassengers dataset (i.e., it doesn't require a very sophisticated model to get near perfect forecasting performance on this dataset).

```
models = [
    NHITS(h=12, input_size=24, scaler_type="robust"),
    RNN(h=12, input_size=24, scaler_type="robust"),
]
nf = NeuralForecast(models=models, freq="M")
nf.fit(df=Y_train_df)
forecasts = nf.predict()
```

```
fig, axes = plt.subplots(2, sharex=True)
plot_series(AirPassengersPanel, forecasts, xlabel="", ylabel="",
    palette="black_and_2color", rm_legend=False, ax=axes)
axes[-1].set(xlabel="Month [1M]")
fig.suptitle("Number of passengers for different airlines", x=.54)
fig.supylabel("Passengers")
fig
```

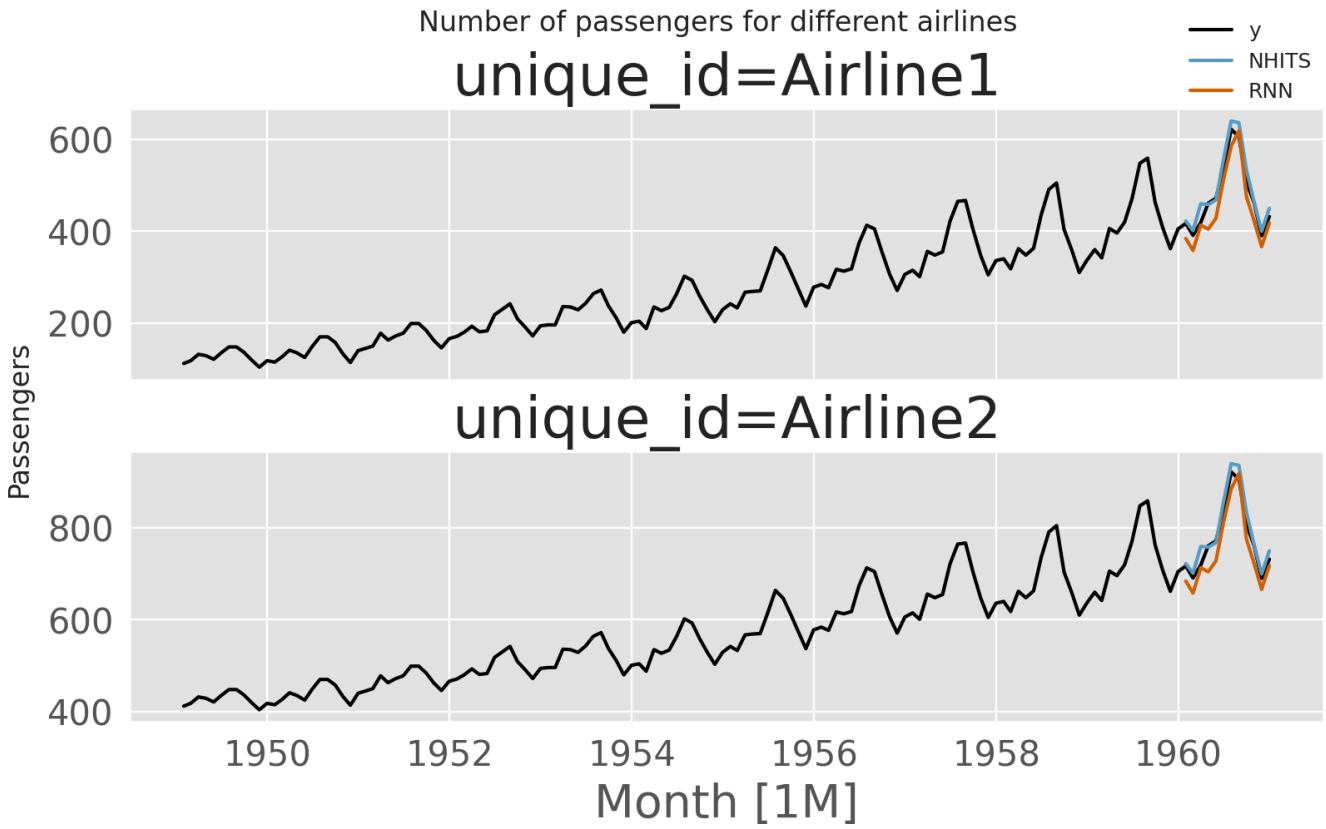

Figure 14.4: AirPassengers NHITS and RNN example.

14.4 Scaling the data

In the AirPassengers example, we included `scaler_type='robust'`. This setting controls how data gets scaled before entering the neural network. Scaling becomes necessary because input ranges can vary widely, which may cause performance issues during training with gradient descent. Common scaling options include:

- `identity` : no scaling
- `standard` : scaled by mean and standard deviation
- `robust` : a variation of standard scaling that handles outliers better. It removes the median and scales the data according to interquartile range (IQR). The IQR is the range between the 1st quartile (25th quantile) and the 3rd quartile (75th quantile).

Each technique scales time series and exogenous covariates based on the t values observed in the input window.

Let's look at our MLP example when using `identity` scaling (i.e., no scaling at all). We can visually verify that the forecast has worsened, and it seems there is a bias present in the forecast.

```
model = MLP(h=12, input_size=24, scaler_type="identity")
nf = NeuralForecast(models=[model], freq="M")
nf.fit(df=Y_train_df)
forecasts = nf.predict()
```

```
fig, axes = plt.subplots(2, sharex=True)
plot_series(AirPassengersPanel, forecasts, xlabel="", ylabel="",
    palette="black_and_blue", rm_legend=False, ax=axes)
axes[-1].set(xlabel="Month [1M]")
fig.suptitle("Number of passengers for different airlines", x=.54)
fig.supylabel("Passengers")
fig
```

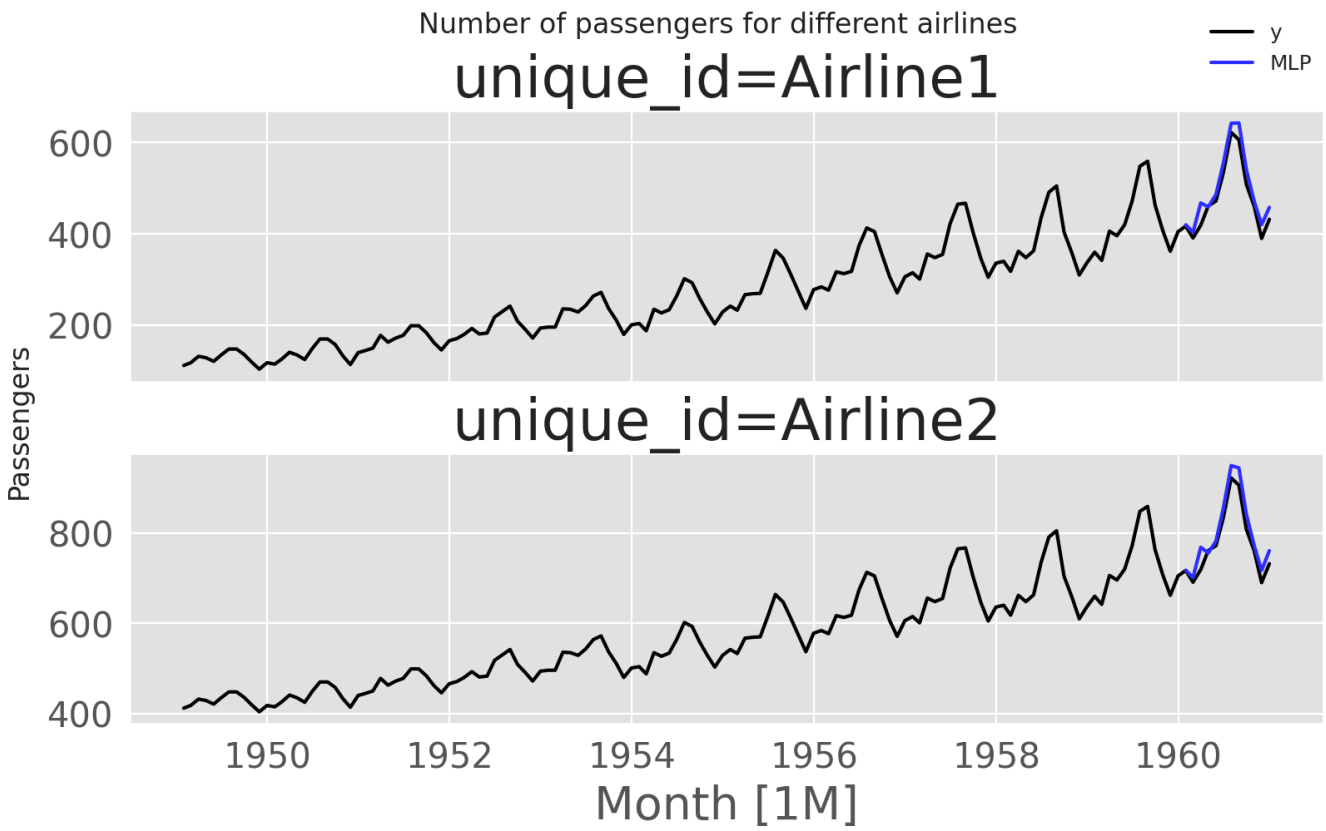

Figure 14.5: AirPassengers MLP example using identity scaling.

14.5 Optimization objectives

When forecasting with neural networks, we aim to learn a function $f_\theta : X \mapsto Y$, using a ground truth Y, such that we can generate forecasts $\hat{Y}$, i.e. $f(X) = \hat{Y}$. Ideally, there is no difference between the ground truth and the forecast: a perfect forecast without forecast error. Of course, with real data, this is impossible.

We measure the difference between the ground truth and the forecast using a *loss function*, which is our *optimization objective*. For example, we can define the *Mean Absolute Error*:

$$\text{MAE} = \frac{1}{N}\sum_{i=0}^{N}|Y_i - \hat{Y}_i|$$

A perfect forecast has a zero MAE, and a very bad forecast a high MAE. Consequently, minimizing the MAE by adjusting the weights of our neural network will improve the forecasting performance.

There are many different optimization objectives. By default, *NeuralForecast* uses MAE as loss function. However, we can also use the *Mean Squared Error* (MSE):

$$\text{MSE} = \frac{1}{N}\sum_{i=0}^{N}(Y_i - \hat{Y}_i)^2$$

The MSE is more susceptible to outliers as a result of the quadratic term and should commonly be used with scaled data. Let's see how our AirPassengers example looks like when using MSE as loss function.

Note that minimizing MSE leads to forecasts equal to the mean of the forecast distribution, while minimizing MAE leads to forecasts equal to the median of the forecast distribution.

```
model = NHITS(h=12, input_size=24, loss=MSE(), scaler_type="robust")
nf = NeuralForecast(models=[model], freq="M")
nf.fit(df=Y_train_df)
forecasts = nf.predict()
```

```
fig, axes = plt.subplots(2, sharex=True)
plot_series(AirPassengersPanel, forecasts, xlabel="", ylabel="",
    palette="black_and_blue", rm_legend=False, ax=axes)
axes[-1].set(xlabel="Month [1M]")
fig.suptitle("Number of passengers for different airlines", x=.54)
fig.supylabel("Passengers")
fig
```

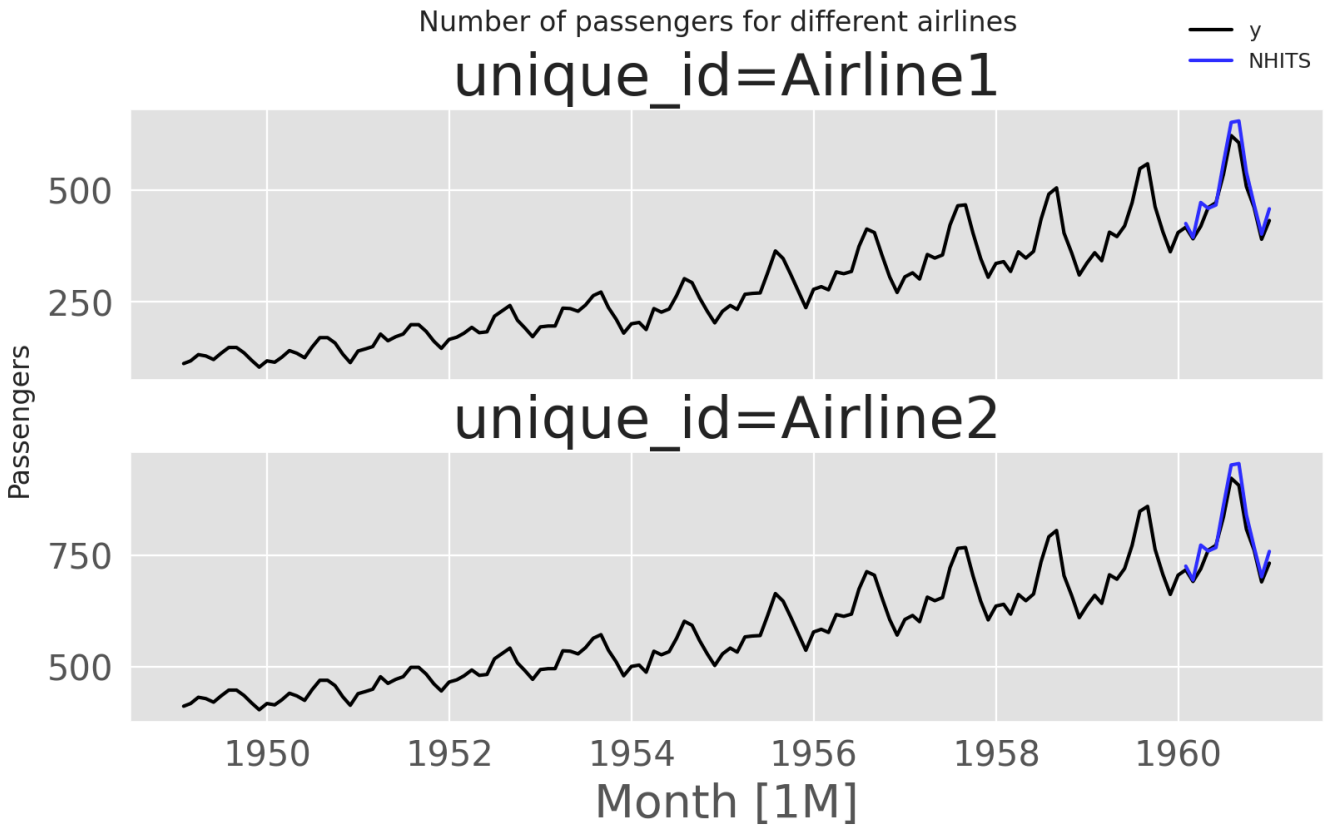

Figure 14.6: AirPassengers NHITS trained with MSE example.

Probabilistic optimization objectives

In the previous example, we showed how to generate *point predictions*. By changing the optimization objective, however, we can also generate probabilistic forecasts, in order to model the full conditional distribution of our forecasting task (rather than only the median (using MAE as loss function) or the mean (using MSE as loss function)).

For example, we can assume that our forecast distribution follows a Gaussian ('Normal') distribution by specifying a `DistributionLoss`. As can be seen, this brings the benefit that we now have a notion of uncertainty around the forecast, indicated by the 80- and 90- percentile shaded regions in the plot.

```
model = NHITS(
    h=12,
    input_size=24,
    loss=DistributionLoss(distribution="Normal"),
    scaler_type="robust",
)
nf = NeuralForecast(models=[model], freq="M")
nf.fit(df=Y_train_df)
forecasts = nf.predict()
```

▶ Code

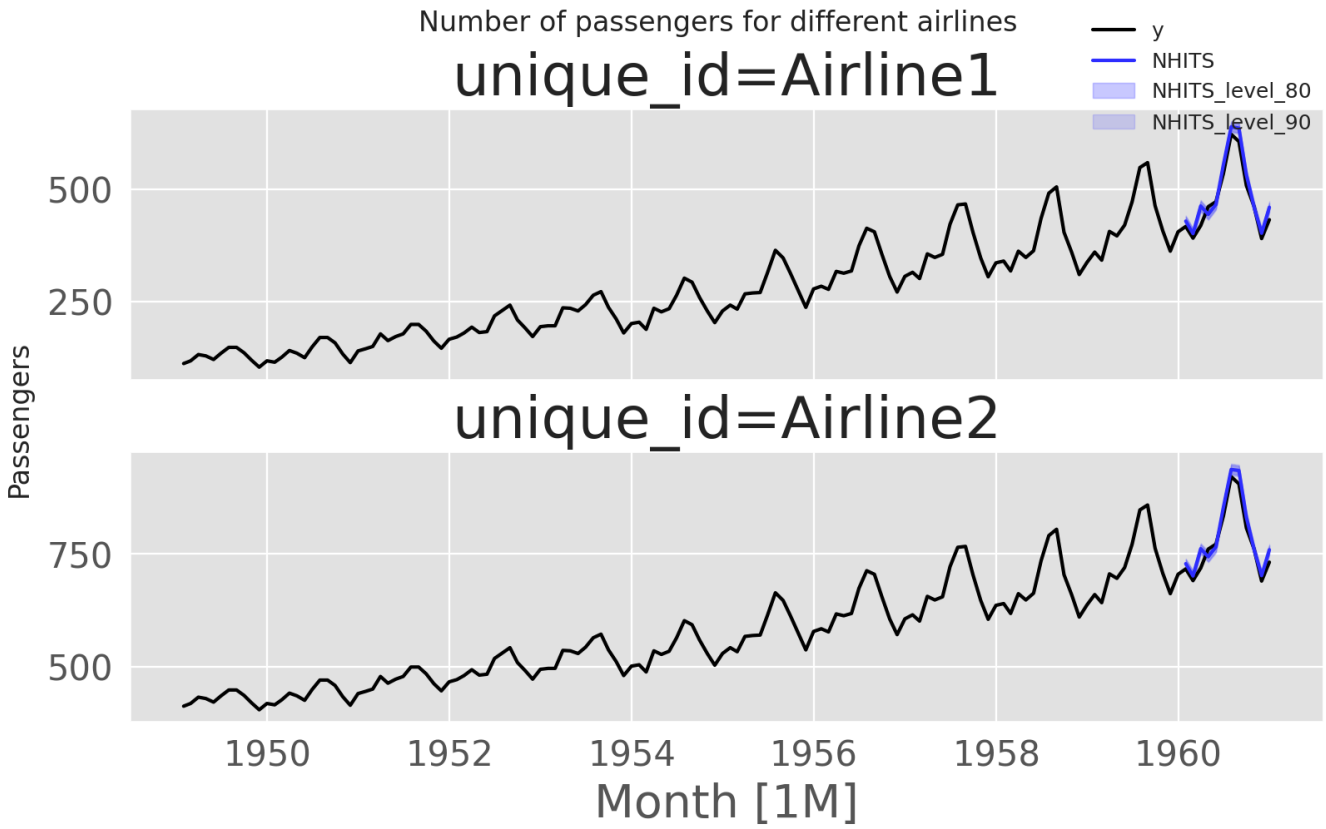

Figure 14.7: AirPassengers NHITS trained with Gaussian DistributionLoss.

The prediction intervals shown in Figure 14.7 are generated by NHITS, which directly predicts future values and their variances in a single pass since we are using `DistributionLoss(distribution="Normal")`. However, this approach can lead to underestimated prediction intervals when the model overfits the training data. Since the model learns distribution parameters based on the training data, narrow intervals may result from the model capturing patterns too closely without properly accounting for future uncertainty.

Unlike some traditional statistical models, where uncertainty increases over time through recursive forecasting, neural models like NHITS rely on what they learn about the data's distribution during training. When the training data has relatively low variability, this can further contribute to narrow prediction intervals.

14.6 Exogenous variables

A key benefit of neural networks is that we can add all sorts of different inputs to the forecasting model. For example, rather than only the values of the time series themselves, we can also provide the model with other sources of information that may be beneficial for our forecast. These *exogenous covariates* can be typically categorized into three distinct types:

1. *Static covariates*: covariates that are static throughout the entire forecast horizon. For example, the identifiers `Airline1` and `Airline2` in our AirPassengers dataset.
2. *Historic covariates*: covariates for which only the historic data is available. For example, the outside temperature can be a historic covariate for a task of forecasting ice cream demand: we don't know the future outside temperature.
3. *Future covariates*: covariates for which we know their future values. For example, the day of the week can be used as a future covariate when trying to predict retail demand: we know the day of the week in advance when making forecasts for a particular day.

In NeuralForecast, each of these three types can be separately specified.

```
# Read the data
df = pd.read_csv("../data/EPF_FR_BE.csv", parse_dates=["ds"])
static_df = pd.read_csv("../data/EPF_FR_BE_static.csv")
futr_df = pd.read_csv("../data/EPF_FR_BE_futr.csv", parse_dates=["ds"])
# Specify the model
horizon = 24  # day-ahead daily forecast
model = BiTCN(
  h=horizon,
  input_size=5 * horizon,
  futr_exog_list=["gen_forecast", "week_day"], # Future exogenous variables
  hist_exog_list=["system_load"],          # Historical exogenous variables
  stat_exog_list=["market_0", "market_1"],     # Static exogenous variables
  scaler_type="robust",
  max_steps=300,                  # Limit max_steps to control compute time
)
# Instantiate the forecaster
nf = NeuralForecast(models=[model], freq="H")
# Fit the model to the training set
nf.fit(df=df, static_df=static_df)
# Generate predictions
forecasts = nf.predict(futr_df=futr_df)
```

```
fig, axes = plt.subplots(2, sharex=True)
plot_series(df, forecasts, max_insample_length=100, xlabel="", ylabel="",
    palette="black_and_blue", rm_legend=False, ax=axes)
axes[-1].set(xlabel="Hour [1H]")
fig.suptitle("Electricity price for different markets", x=.54)
fig.supylabel("Price")
fig
```

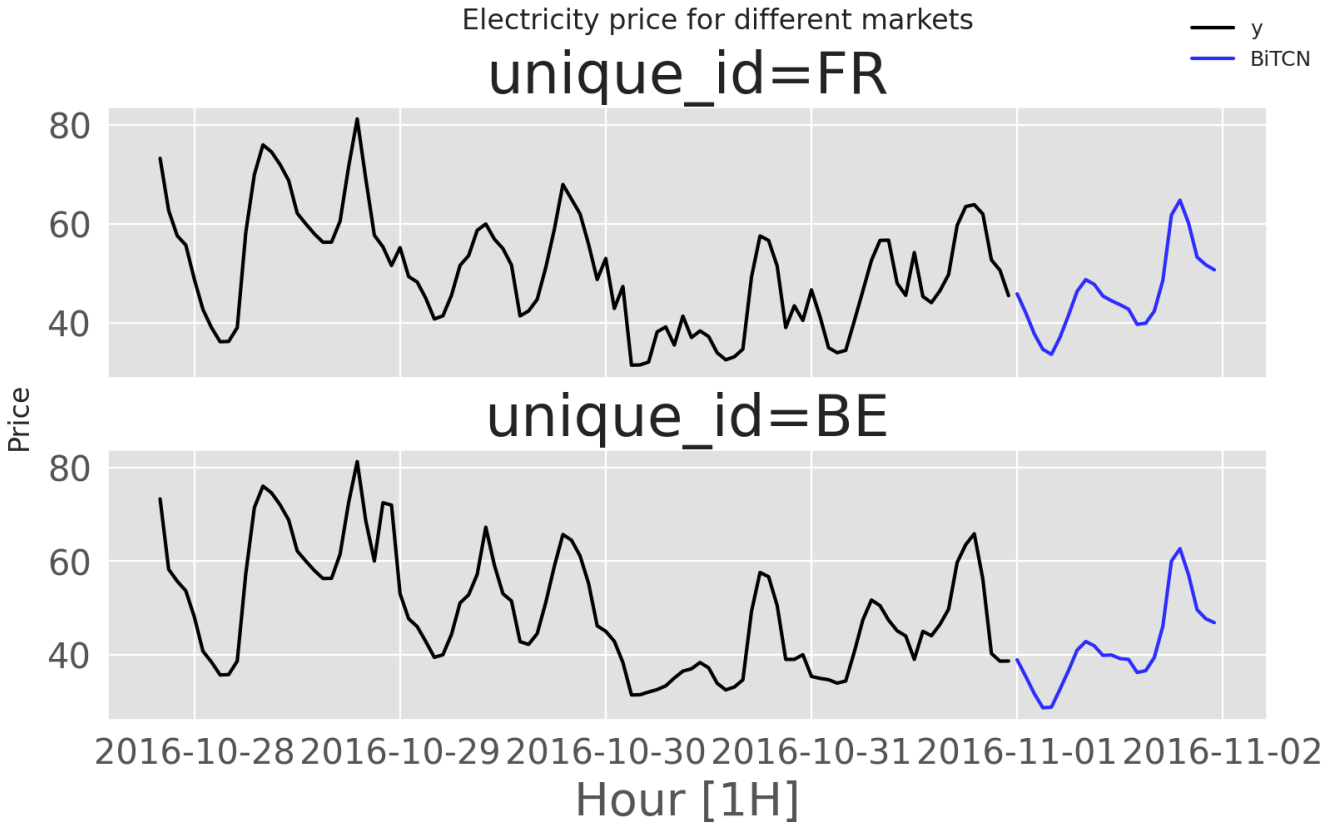

Figure 14.8: BiTCN trained with exogenous variables.

14.7 Hyperparameter optimization

Forecasting with neural networks often requires a practitioner to choose certain settings for training the forecasting model. These settings are commonly referred to as hyperparameters. We've seen a few of them already:

- the number of hidden layers and number of nodes in an MLP;
- the optimization objective;
- the input size (i.e., the amount of historic timesteps from a time series that the neural network considers for creating the forecast);
- the scaler;
- and so forth.

Idealmente, um profissional não precisa escolher essas configurações manualmente, mas sim deixar que um algoritmo de otimização as selecione. Para isso, podemos realizar *a otimização de hiperparâmetros* . *O NeuralForecast* pode realizar a otimização de hiperparâmetros automaticamente com base em uma configuração padrão. Para isso, ele utiliza modelos *Automáticos* , que são versões dos modelos da Tabela 14.1 que permitem otimizar automaticamente os hiperparâmetros da rede neural. Por um número específico de iterações, um modelo *Automático* selecionará de forma inteligente um conjunto de hiperparâmetros

dentro do intervalo elegível e os avaliará em um conjunto de validação. Ao final do processo de treinamento, os hiperparâmetros com melhor desempenho são retornados e podem ser usados na aplicação de previsão subsequente.

```
test_mask = AirPassengersPanel["ds"] >= "1960"
Y_train_df = AirPassengersPanel[~test_mask]
Y_test_df = AirPassengersPanel[test_mask].reset_index(drop=True)

# Get the default hyperparameter range for NHITS.
# Update random_seed and n_pool_kernel_size with custom values.
# Also limit max_steps to control compute time.
nhits_config = {
    **AutoNHITS.get_default_config(h=12, backend="ray"),
    "random_seed": tune.randint(1, 10),
    "n_pool_kernel_size": tune.choice([[2, 2, 2], [16, 8, 1]]),
    "max_steps": tune.choice([100]),
}
# Instantiate the AutoNHITS model with the chosen configuration.
# We optimize for 12 steps
# i.e. we evaluate 10 different hyperparameter configurations.
model = AutoNHITS(h=12, num_samples=10, config=nhits_config)
# Instantiate the forecaster
nf = NeuralForecast(models=[model], freq="M")
# Fit the model to the training set
nf.fit(df=Y_train_df)
# Generate predictions
forecasts = nf.predict()
```

```
fig, axes = plt.subplots(2, sharex=True)
plot_series(AirPassengersPanel, forecasts, xlabel="", ylabel="",
    palette="black_and_blue", rm_legend=False, ax=axes)
axes[-1].set(xlabel="Month [1M]")
fig.suptitle("Number of passengers for different airlines", x=.54)
fig.supylabel("Passengers")
fig
```

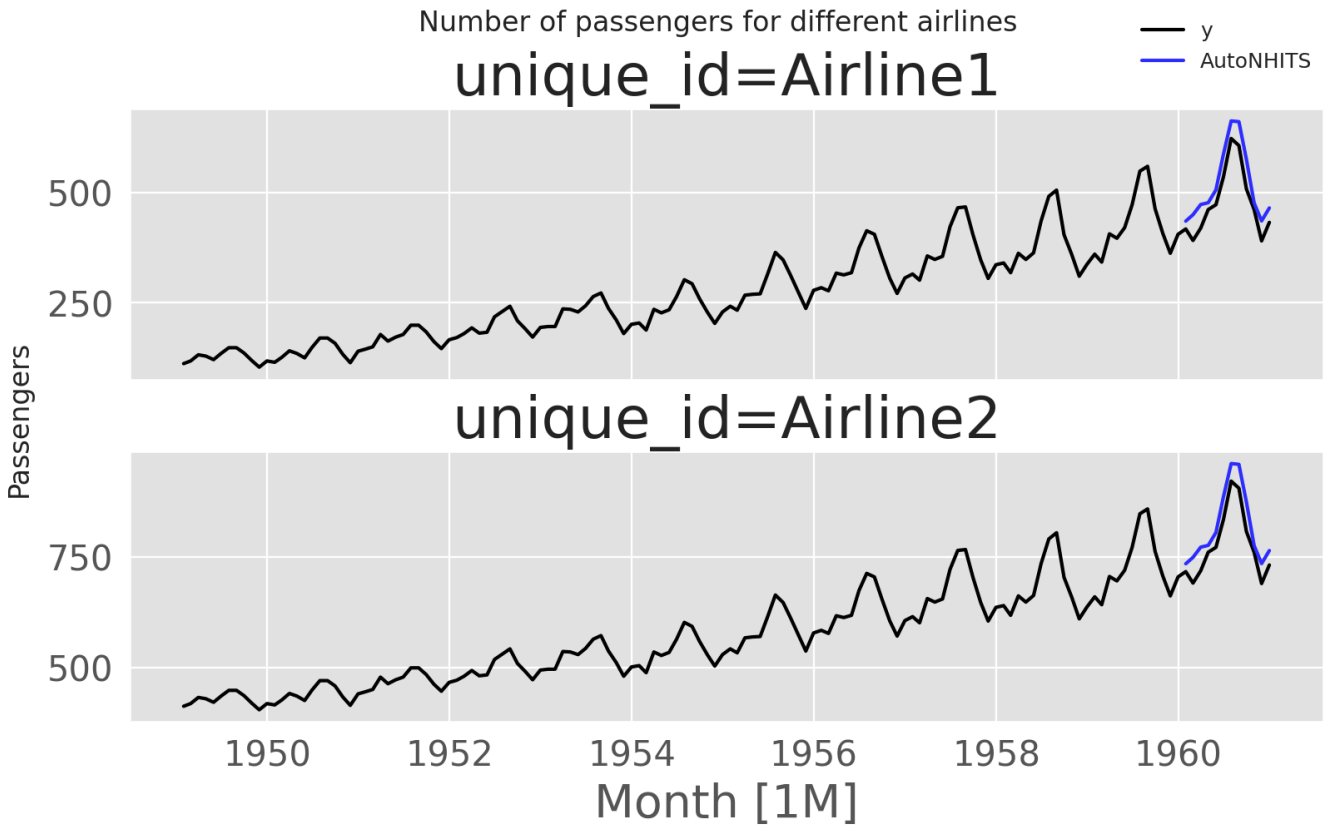

Figura 14.9: AirPassengers NHITS com otimização automática de hiperparâmetros.

14.8 Exercícios

1. Utilize o modelo LSTM para prever os próximos 24 passos para todas as séries do `pedestrian` conjunto de dados.

2. Considere os dados semanais sobre o fornecimento de produtos de gasolina automotiva acabados nos EUA (milhões de barris por dia) (série `us_gasoline`).

   a. Utilize o modelo MLP para prever os dados das próximas 13 semanas.
   b. Repita o exercício anterior usando os modelos AutoMLP e AutoNHITS do NeuralForecast.
   c. Compare a precisão dos modelos que você gerou.
   d. Compare a precisão dos modelos com os modelos do Capítulo 12, Exercício 2.

14.9 Leitura complementar

- Bengio, Courville e Vincent (2013) fornecem uma visão geral da aprendizagem de representação.

- Mariet e Kuznetsov (2019) para uma introdução mais formal à modelagem de sequência para sequência para séries temporais.
- Hewamalage, Bergmeir e Bandara (2021) para uma revisão sobre redes neurais recorrentes.

14.10 Módulos e classes utilizados

NeuralPrevisão

- `NeuralForecast` Classe - Mecanismo principal de previsão para modelos de redes neurais
- `MLP()` Modelo - Arquitetura básica de perceptron multicamadas
- `NHITS()` Modelo - Interpolação hierárquica neural para séries temporais
- `RNN()` Modelo - Arquitetura de rede neural recorrente
- `AutoNHITS()` Modelo - Otimização automática de hiperparâmetros para NHITS

Previsão de utilidades

- `plot_series()` Função - Para visualizar dados de séries temporais e previsões.

Previsão de conjuntos de dados

- `AirPassengersPanel` Conjunto de dados - Conjunto de dados clássico de passageiros de companhias aéreas para exemplos

Publicado pela OTexts usando Quarto . © 2026